

Overlapping MoE Synchronization and Data Transfer with In-Network Computing

主线：在具有入口与尾部协调的 EP 路径上，将 READY 汇总与数据上传重叠，并在目标数据输出完成后生成有序 DONE，减少控制等待在关键路径上的暴露。

1. Introduction — 引言

- **问题：**小 batch 缩短了 payload 阶段，但跨 rank 协调的成本不会等比例下降。
- **研究场景：**DeepEP V2 跨机 Direct Dispatch/Combine，具有入口 barrier 和发送收尾之后的全 EP 完成信号交换。
- **方法：**rank 本地就绪后上传至 INC 暂存；INC 汇总 READY 后放行，并在逐目标尾包后追加 DONE。
- **贡献：**基线同步测量、网内协调设计及其时延分析。引言用一个真实推理结果说明动机，完整数据放在 §5；当前没有 INC 原型加速测量。

2. Background and Motivation — 背景与动机

2.1 Expert-Parallel Communication

说明 Dispatch → expert compute → Combine；定义 token、rank、topk 和路由。路由决定各目标的数据量，通信 staging 提供落点，最终 expert 布局可依赖 count 交换。

2.2 Synchronization Overhead in Small-Batch EP

按执行顺序解释：入口就绪 barrier → DATA 与元数据处理 → TMA 等待、本地汇合、GIN flush/fence → 全 EP 完成 signal 交换。区分接收资源可写、输入完整可见、buffer 可复用三个条件。

主论证针对跨机 Direct。Hybrid、LL、NCCL EP HT 的边界不同，仅用于说明适用范围，不把 Direct 的双同步行为推广到所有 EP 实现。

固定 EP 规模时，payload 随 token 数缩小，控制交换仍面向同一组 rank。例如 EP8 在 $T/\text{rank}=8$ 时前同步约占算子 20%，到 $T=128$ 降至约 5%，绝对时长仍约 15 μs 。这一现象引出“协调能否与 DATA 重叠”的问题。

2.3 Opportunities for In-Network Computing

INC 背景：网内计算允许网络设备在转发过程中维护状态、聚合信息并执行有限计算，已有工作将其用于集合归约和 MoE 通信。本文利用状态汇总、数据暂存和完成通知能力；可编程存储、容量保障与保序通知是实现条件，并非所有交换机天然具备。

前同步机会：本地数据就绪可以早于全体就绪。rank 可先向已预留的网络暂存区上传，INC 同时汇总 READY；目标写入仍等待所需就绪条件及可写资源满足。

后同步机会：INC 掌握目标的预期流量并跟踪输出进度，在尾数据后追加有序完成通知，从而缩短基线中数据收尾后才发起的端点完成交换。观察到通知必须意味着覆盖数据对目标可见；真正的全局依赖仍需保留。

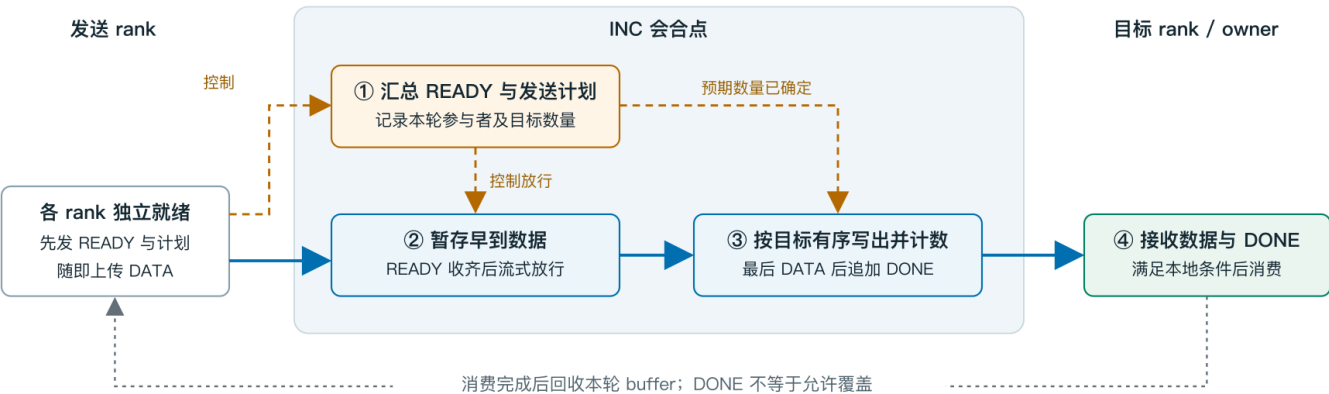
Key Insight：同步条件必须得到满足，但同步通信不必与数据传输串行执行。利用数据路径上的网内计算节点，可以将就绪汇总与提前上传重叠，并将目标完成通知附着于尾数据，从而缩短同步在通信关键路径上暴露的时间。第 3 章展开安全暂存、资源隔离和保序完成的设计。

3. Design — 设计

3.1 Overview and Assumptions

INC：就绪汇总与上传重叠，完成通知随尾包送达

资源前提：预留端点缓冲区和有界网络暂存；本轮可写状态由 READY 确认



Dispatch: 转发 activation, 可结合组播

Combine: 按 contributor 集合归约, 再有序写回结果

INC 节点维护本轮参与者、逐目标流量计划、输出进度、目标内存描述符和有上限的暂存。受其完成判定覆盖的数据必须经过该节点；多 rail 和本地 bypass 需要明确的完成域与汇合。

上传前获得网络暂存额度；写出前目标本轮 staging 必须可写。实现需要可编程存储和理解传输保序语义的输出接口，现阶段不假定商用交换机仅靠 packet buffer 就能完整承载。

3.2 Readiness Aggregation and Early Upload

rank 本地就绪后发送 READY 和流量计划，随后上传已准入 DATA。INC 缓存早到数据，所需 READY 和目标写入条件满足后，按出口能力放行。就绪依赖仍存在，但源到 INC 的传输可以提前进行。

Dispatch 的 count/layout 处理可与写入已预留 staging 重叠；组播可结合使用，其带宽收益不作为同步贡献。

3.3 Completion Notification

每个 source 封定本轮发往目标的唯一记录数，零流量也需显式说明。INC 在完整输出后追加 DONE，并保证：

目标观察到 DONE ⇒ 本轮覆盖的全部数据已对目标 GPU 可见。

计数必须排除重传重复；跨 QP/rail 的完成域需要汇合。逐目标 DONE 仅在目标本地输入完成足以释放后继计算时替代全局尾协调；真正的全局依赖仍保留。

Combine 复用 Dispatch 的 contributor 集合，按 route-slot 区分贡献，执行既有网内归约，再跟踪 reduced result 的输出完成。归约算术及节省流量的机制沿用前作。

3.4 Buffer Management and Progress

generation/epoch 隔离各轮资源。DONE 允许消费；最后一个消费者结束后才允许复用。发送端仍需保证 transport 不再读取源 buffer。

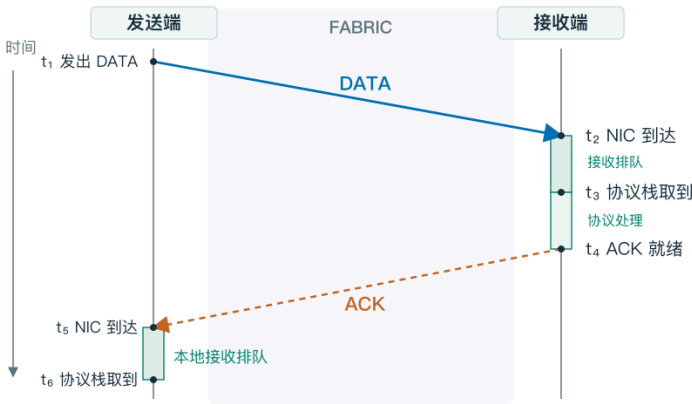
容量覆盖早到数据、进出速率失配和归约 accumulator。容量不足时同路背压；READY 等控制消息必须有独立进展保证。PFC 不能替代容量准入与死锁分析，也不保证压力下吞吐不变。

4. Latency Analysis — 时延分析

4.1 Delay Model

时延分解：把端点处理与网络传输分开

基于 Swift Fig. 2 重绘 · 六个事件时间戳，解释一轮 DATA / ACK 的往返



端点时延

$$D_{\text{endpoint}} = (t_4 - t_2) + (t_6 - t_5)$$

网络时延（残差）

$$D_{\text{fabric}} = (t_6 - t_1) - D_{\text{endpoint}}$$

映射到 EP 的观察位置

发送准备与汇合 / fabric 传输
目标可见性与 signal 观察

借鉴 Swift 的端点/fabric 分解，定义本地就绪、DATA 注入、目标数据可见和消费者释放时刻。实测 barrier 包含端点处理和等待，不能直接等同于网络 RTT。

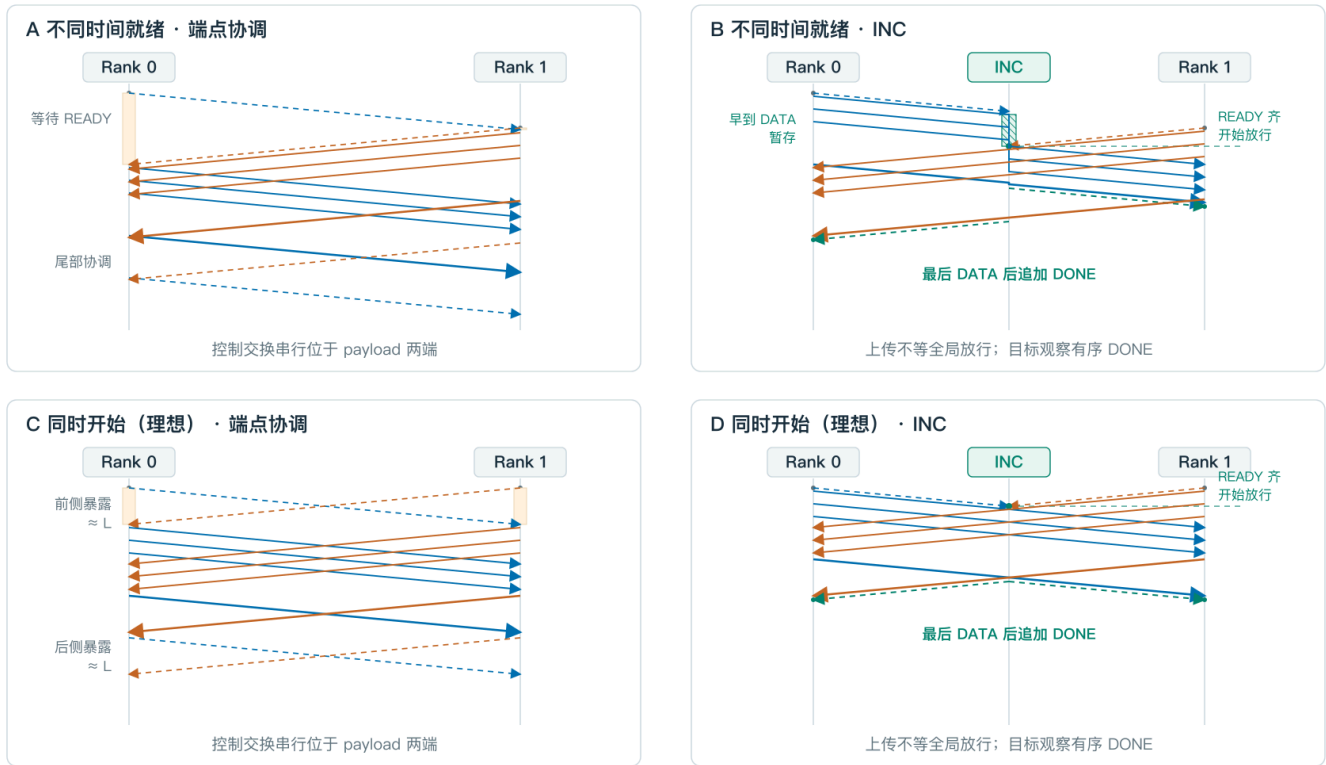
4.2 Readiness Overlap

先讨论各 rank 不同时就绪的一般情况：基线等自己与 peer 就绪后才发送，INC 允许已准入的源先上传，目标写入仍等 READY 和目标资源到齐。暂存、慢 rank 和出口服务共同决定完成时间，不能将提前上传量直接换算为加速。

INC 的两项收益：提前上传，随尾包通知完成

上排：一般到达情况 下排：理想参考 颜色区分两个发送方向，线型区分数据与控制

—▶ 实线：DATA - - - - -▶ 虚线：READY / 完成 signal - - - - -▶ 绿色尾标：DONE ▨ 斜线区：暂存等待



理想参考： $L = RTT_{fabric} / 2$ ；前后各约减少 L ，另计 INC 开销。一般情况由就绪偏斜、尾部暴露与出口服务共同决定。

4.3 Completion Overlap

将发送端收尾完成、目标数据可见、完成通知被观察作为不同事件。对比同一目标的释放时刻：

尾部净收益 = 基线消费者释放时间 - INC 消费者释放时间。

窄 signal 窗口并未直接测得“数据可见之后仍暴露的等待”。新增通知、保序、排队或确认往返都要计入成本。同步到达且两边各有一次单向控制传播的理想特例，机会约为一个 fabric RTT 减去新增开销；只作为参考，不代替一般路径分析。

5. Evaluation — 实验评估

本章测量基线的同步开销及其在推理中的影响；INC 的端到端收益仍需实现后评估。

5.1 Experimental Setup and Methodology

- H200：两台各八卡，机内 NVSwitch，机间八轨 400GbE RoCE；跨机使用 GDAKI3/TC162，DeepEP V2 Direct。
- 微基准：H=7168、topk=8、E=256、BF16、balanced/uniform。每档三次独立进程、每次 200 次有效迭代；后同步 T=32 合并两批共六轮。
- 每次迭代取该阶段耗时最短的 rank；占比除以该 rank 同次 trace-on 算子时长。均值误差表示轮间标准差，不把不同 rank 的阶段占比相加。
- 总算子时延来自 trace-off，阶段来自 trace-on。推理的 GPU-step 占比使用同 rank/step 配对并跨 rank 平均，与微基准 rank-min 统计不同。

- 实测使用通信插桩；推理还含 expert-padding 正确性修复。所有数值均为已有软件基线观测。

5.2 Entry Synchronization

主图同时展示前同步绝对时长和算子占比。EP8 的前同步维持约 15 μ s，占比随 T 增大而下降。

T/rank	Dispatch 前同步占比	Combine 前同步占比
8	(20.57 $\pm$ 0.06)%	(19.84 $\pm$ 0.13)%
32	(12.25 $\pm$ 0.17)%	(12.26 $\pm$ 0.26)%
64	(8.29 $\pm$ 0.51)%	(8.20 $\pm$ 0.16)%
128	(4.88 $\pm$ 0.04)%	(5.05 $\pm$ 0.19)%

各固定 EP 配置下对可用 T 档作常数描述；所有列出的点参与估计，不再称为独立预测验证。

EP 卡数	参与估计的 T/rank	Dispatch (μ s)	Combine (μ s)	各档均值最大偏差 (μ s)
4	8/32/96/128	11.65	11.54	0.06
8	8/32/64/96/128	14.96	14.94	0.16
16	8/32/96/128	18.36	18.28	0.09

EP8 的 T=64 是后续不同插桩批次补测。不同 EP 配置也改变每机卡数与网络并发，不据此外推任意 rank 数的规律。
来源： [配对数值](#)、[前同步常数记录](#)；[主图源码](#)。

5.3 Completion Signaling

计时从本地完成 signal 发起到全部预期 signal 被观察到，排除此前的 flush。保留轮间波动，不再展示宽 post 拟合。

T/rank	Dispatch signal (μ s)	Combine signal (μ s)
8	13.97 $\pm$ 0.04	13.56 $\pm$ 0.24
16	13.98 $\pm$ 0.23	13.78 $\pm$ 0.18
32	11.67 $\pm$ 0.94	11.70 $\pm$ 1.43
64	11.81 $\pm$ 0.42	11.76 $\pm$ 0.29
96	11.12 $\pm$ 0.22	11.24 $\pm$ 0.92
128	11.73 $\pm$ 0.07	11.88 $\pm$ 0.18

T=8/16 约为 14 μ s，已测 T $\geq$ 32 约为 11–12 μ s；T=32 的慢轮次全部保留。固定 EP8 时信号数量不随 T 改变，但窗口仍可能受 peer 就绪差、端点和网络排队影响。尚无证据支持协议阈值或通用时延公式，也不能将窗口全部计为 INC 收益。来源： [复测与代码分析](#)；[主图源码](#)。

5.4 Synchronization in MoE Inference

Qwen3–30B–A3B, BF16, EP8 Direct, decode 输入/输出各 128 token。比较同一 EP 规模和并发下的单机与跨机前同步占比。

拓扑	并发	实测 token/rank	D+C 前同步占 GPU step
单机 1n8	64	8	9.33%
跨机 2n4	64	8	16.72%
单机 1n8	128	16	7.94%
跨机 2n4	128	16	15.19%

跨机约 15%—17%，单机约 8%—9%。这是采样 GPU step 的占比，不是请求延迟或 INC 加速上限；三次 client 窗口复用服务实例，未测真实模型中窄完成 signal 的逐层占比。来源：[端到端数值](#)、[前同步分项](#)；[主图源码](#)。

6. Related Work — 相关工作

6.1 EP Communication

[DeepEP](#)、[NCCL EP](#) 提供基线及不同同步组织；[SwiftEP](#)、[UEP](#)、[Perseus](#)、[UBEP](#) 分别从传输、可移植接口或流水内同步等方向优化 EP。比较其处理的依赖与本文的网络 READY 汇总、暂存放行、目标完成通知，避免反复逐篇否定。

6.2 In-Network Computation

[DySHARP](#)、[MultiWrite](#) 提供 MoE multicast/reduction 与写语义；[SHARP](#)、[SwitchML](#) 提供集合卸载与归约状态管理先例。本文借用这些数据面能力，贡献集中在协调时序。

6.3 Synchronization Offload

[EPIC](#) 提供 INC 协议与资源抽象；[GPU-Initiated Networking](#) 提供 PUT/signal/fence 等端点原语。比较控制状态放置位置及完成语义。Swift 和测量工具在相应方法处引用，不另设一组重复介绍。

7. Conclusion — 结论

跨机 Direct 的同步成本在小 batch 下占比显著。本文提出通过网内就绪汇总和有序完成通知重叠两端协调，并给出所需的可见性、资源与进展条件。现有实验支撑问题动机，实际收益仍需 INC 实现验证。